

Notification Service

Thông báo theo sự kiện và realtime

Nhóm bài tập lớn Công nghệ Web

Cập nhật: 26/05/2026

1. Vai trò

Notification Service chạy tại cổng 4000. Dịch vụ nhận yêu cầu tạo thông báo từ các thao tác nghiệp vụ, lưu lịch sử thông báo theo từng người dùng vào Redis, đồng thời phát sự kiện Socket.IO cho trình duyệt đang kết nối.

2. Thông báo bắt nguồn từ đâu

Trong hệ thống hiện tại, các yêu cầu tạo thông báo quan trọng bắt nguồn từ Order Service, sau khi dữ liệu đơn hàng đã thay đổi thành công:

- Tạo đơn mới: gửi cho buyer và seller của sản phẩm.
- Seller chấp nhận đơn: gửi cho buyer và các tài khoản admin.
- Buyer hủy đơn: gửi cho seller và admin.
- Seller từ chối đơn hoặc gửi bằng chứng hoàn tiền: gửi cho buyer.
- Admin chuyển sang đang giao hoặc đã giao: gửi cho buyer.

3. Luồng gửi từ Order Service đến người dùng

3.1. Bước 1: Order Service tạo nội dung

Sau khi insert hoặc update order trên Supabase thành công, hàm nghiệp vụ trong Order Service gọi hàm gửi thông báo với các trường:

- toUserId: email/định danh người sẽ nhận.
- type: loại sự kiện, ví dụ ORDER_CREATED, ORDER_STATUS_CHANGED.
- title, body: nội dung hiển thị.
- data: channel, targetUrl, orderId, shopId, status.

3.2. Bước 2: Gửi request sang Notification Service

Order Service thực hiện HTTP POST tới:

`http://notification-service:4000/notification-service/publish`

Trong chạy local không container, URL có thể là `http://localhost:4000/notification-service`. Khi chạy Docker, phải dùng tên container `notification-service`; nếu dùng `localhost`, Order Service sẽ tự gọi vào container của chính nó và log `Failed to publish notification: fetch failed`.

3.3. Bước 3: Notification Service nhận và chuẩn hóa dữ liệu

Endpoint /publish nhận body, bắt buộc phải có toUserId. Service bổ sung id, loại mặc định, thời gian tạo và đảm bảo payload có cấu trúc thống nhất trước khi xử lý tiếp.

3.4. Bước 4: Đưa thông báo qua RabbitMQ

Thông báo chuẩn hóa được ghi vào exchange notifications với routing key mặc định notification.created. Queue notification_service_queue nhận message. Cách này giúp request từ Order Service kết thúc nhanh sau khi thông báo đã được tiếp nhận, không phải chờ toàn bộ việc lưu và đẩy ra giao diện.

Nếu RabbitMQ tạm thời không dùng được lúc request đến, controller có nhánh dự phòng: lưu thông báo thẳng vào Redis để không làm mất nội dung cần hiển thị.

3.5. Bước 5: Lấy message từ queue và lưu Redis

Luồng xử lý chạy nền trong Notification Service đọc từng message từ queue. Với mỗi message hợp lệ, service lưu JSON vào đầu danh sách Redis:

```
notifications:{userId}
```

Sau đó danh sách được cắt lại để giữ tối đa số lượng cấu hình bởi NOTIFICATION_MAX_ITEMS, mặc định là 50. Vì vậy hệ thống chỉ giữ các thông báo gần đây nhất cho từng người dùng.

3.6. Bước 6: Đẩy realtime về frontend

Sau khi lưu Redis thành công, Notification Service phát sự kiện socket tên notification tới đúng toUserId. Người dùng đang mở trình duyệt có thể nhận thông tin ngay mà không cần tải lại trang.

3.7. Bước 7: Frontend đọc danh sách lịch sử

Trang thông báo hoặc dashboard gọi:

```
GET /notification-service/read-notifications
?userId=<nguo-i-nhan>&channel=<buyer|seller|admin>&page=1&limit=10
```

Service đọc list Redis, chuyển JSON về object, lọc theo data.channel, tính phân trang và trả về frontend. Khi người dùng bấm thông báo, frontend dùng data.targetUrl để mở đúng trang đơn hàng hoặc trang quản lý shop.

4. Ví dụ luồng khi seller chấp nhận đơn

1. Seller bấm chấp nhận đơn tại giao diện quản lý cửa hàng.

2. Request đi qua Nginx, Gateway tới Order Service.
3. Order Service kiểm tra seller và trạng thái pending, sau đó cập nhật đơn thành processing.
4. Order Service gửi một thông báo cho buyer rằng đơn đã được chấp nhận.
5. Order Service tìm các admin và gửi thông báo rằng có đơn sẵn sàng cho quy trình giao hàng.
6. Notification Service lưu từng thông báo theo key Redis của từng người nhận và phát socket nếu người đó online.
7. Buyer hoặc admin tải trang Notifications sẽ đọc được thông báo tương ứng theo channel.

5. Tại sao thiết kế như vậy

- Order Service chỉ quyết định khi nào và gửi nội dung gì; Notification Service chịu trách nhiệm lưu và phát thông báo. Hai phần nghiệp vụ không bị trộn lẫn.
- RabbitMQ làm vùng đệm cho thao tác thông báo, phù hợp khi một hành động phải gửi cho nhiều người nhận.
- Redis phù hợp cho danh sách ngắn, đọc nhanh và lọc đơn giản theo vai trò.
- Socket.IO mang lại cập nhật realtime; API đọc Redis vẫn bảo đảm người dùng xem được lịch sử khi mở trang sau đó.

6. Kiểm tra khi không thấy thông báo

1. Kiểm tra log Order Service: không được còn lỗi Failed to publish notification.
2. Kiểm tra biến NOTIFICATION_SERVICE_URL trong Compose trỏ đúng service nội bộ.
3. Kiểm tra log Notification Service có kết nối Redis và đã kết nối RabbitMQ.
4. Gọi endpoint đọc với đúng userId và đúng channel; nếu channel khác, thông báo tồn tại nhưng bị lọc khỏi tab đang xem.
5. Không xóa volume Redis nếu cần giữ thông báo cũ sau khi khởi động lại stack.